LINMA2474: Project 1

Graph learning and image inpainting with Riemannian optimization

1 Context

Learning a graph from data has become of increasing interest over the last few years, namely since the emergence and the success of both graph signal processing (GSP) [1] and graph neural networks (GNN) [2]. The idea is to discover hidden structure in the data by learning correlations among datapoints, which are then interpreted as the nodes of a graph. Modeling data as a graph allows us to implement algorithms that exploit the structure underlying the data. In this project, we will use the learned graph to regularize a low-rank image inpainting algorithm.

The project will be divided in two parts.

Part 1: Given a data matrix $X \in \mathbb{R}^{n \times m}$, where n is the dimension of datapoints (number of features characterizing the data) and m is the number of samples, we construct a graph with n vertices, whose edges represent the strength of the correlation between two features in the dataset. In this project, a graph is represented by its Laplacian matrix. We consider a simplified version of the undirected/symmetric graph learning problem:

$$\min_{L \in \mathcal{L}} f(L) = \operatorname{tr}(X^\top L X), \tag{1}$$

where $\mathcal{L} := \{L \in \mathbb{R}^{n \times n} : L1_n = 0, \operatorname{tr}(L) = n, L \succeq 0\}$ is our set of feasible Laplacian matrices (this formulation is adapted from [1]; note that, for the sake of the project we do not require all off-diagonal entries to be nonpositive).

In this project, we will solve the graph learning problem (1) for a dataset composed of images. Your goal will be to learn the best Laplacian matrix for three different graphs, corresponding to the three color channels $c \in \{R, G, B\}$. For each graph, the vertices will represent pixel values in the RGB channel considered.

Part 2: In the second part of the project, we will consider the image inpainting problem, aiming at reconstructing an image with several missing entries (see examples in Figure 1). To this end, we will use the information provided by the graph learned in part 1 to regularize our reconstruction algorithm.

2 Dataset and Python toolbox

For this project, we will use the dataset CIFAR-10 composed of 50000 RGB images $3 \times 32 \times 32$. It is available as a zip file at `https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz`.

You should unzip the file in the same working directory as the provided Notebook. We will focus on a subset of frogs as shown in Figure 1 and divide the dataset in two parts:

- X_{part1} containing 200 frog images for Part 1;
- X_{part2} containing 32 images used in Part 2 to generate the corrupted samples.

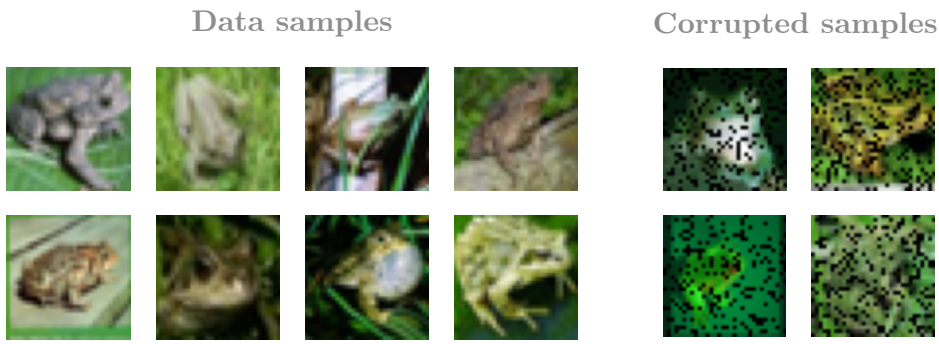

Figure 1: Frog samples of CIFAR-10 (left) and examples of corrupted samples (right).

For the implementation of Riemannian optimization algorithms, you will rely on the `PyManopt` toolbox [3]. The documentation is available at `https://pymanopt.org/docs/stable/`) with several examples in `https://github.com/pymanopt/pymanopt/tree/master/examples`.

3 Project

Part 1: In this first part of the project, we will learn a weighted graph capturing correlations between pixels across the dataset. More precisely, we will decompose each image in three channels (one for each color) and learn a Laplacian matrix $L^{(c)} \in \mathcal{L}$ that represents the graph associated with channel $c \in \{R, G, B\}$. We propose you to flatten the images column by column. Thus, for the channel c of the kth image, the pixel in position (s, t) with $1 \leqslant s, t \leqslant 32$ corresponds to pixel i, given by $i = (s-1) \cdot 32 + t$ such that $1 \leqslant i \leqslant n = 1024 = 32^2$.

In order to obtain a graph that achieves a trade-off between simplicity and data agreement, we will impose some low-rank structure on the Laplacian matrix, and search our candidate Laplacian matrix into the smaller set

$$\mathcal{L}_k := \{L \in \mathbb{R}^{n \times n} : L1_n = 0, \operatorname{tr}(L) = n, L \succeq 0, \operatorname{rank}(L) \leq k\},$$

i.e., we will impose the Laplacian matrix to have rank at most k, for some $k \leq n$. As seen in the first lecture, the set $\mathcal{L}_k$ does not admit a manifold structure, due to the upper bound on the rank. This issue can be alleviated by relying on the Burer-Monteiro factorization of the matrices: any

positive-semidefinite matrix $L \in \mathbb{R}^{n\times n}$ of rank at most k can be represented by a matrix $M \in \mathbb{R}^{n\times k}$ satisfying $L = MM^\top$. This factorization will allow you to rewrite $\mathcal{L}_k$ as

$$\tilde{\mathcal{L}}_k := \{MM^\top \in \mathbb{R}^{n\times n} : M \in \mathbb{R}^{n\times k}, MM^\top 1_n = 0, \mathrm{tr}(MM^\top) = n\}. \tag{2}$$

1. Show that the sets $\mathcal{L}_k$ and $\tilde{\mathcal{L}}_k$ are identical.

2. We now show that (2) allows us to formulate (1) as an optimization problem on the $(nk - k - 1)$-dimensional hypersphere. We ask you to:

   (i) show that if $\mathrm{vec}(M) \in \mathcal{S}^{(n-1)k-1}$, then for any orthogonal basis $Q_\perp \in \mathbb{R}^{n\times(n-1)}$ of $\{1_n\}^\perp$, $L = n(Q_\perp M)(Q_\perp M)^\top$ is an admissible Laplacian, i.e., $L \in \mathcal{L}_k$.

   (ii) construct, in your Python code, an orthogonal basis $Q_\perp \in \mathbb{R}^{n\times(n-1)}$ of $\{1_n\}^\perp$ from a QR decomposition of $[1_n|I_n]$, and fix it for the whole project.

3. Implement the graph learning problem in PyManopt with the `Sphere` manifold as:

$$\min_{M\in\mathbb{R}^{(n-1)\times k}} \mathrm{tr}(X^\top (Q_\perp M)(Q_\perp M)^\top X) \ \text{ s.t. } \mathrm{vec}(M) \in \mathcal{S}^{(n-1)k-1}. \tag{3}$$

   For each channel $c \in \{R, G, B\}$ of the dataset X_{part1}, solve (3) to compute the associated Laplacian matrix representation $M^{(c)}$ by running the `SteepestDescent` and `TrustRegions` optimizers over 100 iterations, with $k = n-1$ and $k = 30$. For the red channel $c = R$, report on a plot the evolution of the cost function with respect to the number of iterations for each algorithm and for both rank configurations. Discuss your results.

4. Analyze the time complexity related to the rank k by examining the time taken by a single iteration of the `SteepestDescent` algorithm with increasing rank k.

5. The retraction implemented in the `Sphere` library is by default $R_v(x) = \frac{x+v}{\|x+v\|}$. We now ask you to replace it by the retraction seen in the second practical session, Exercise 3[1], i.e.,

$$R_v^{\text{new}}(x) = x\ \tfrac{4-\|v\|}{4+\|v\|} + v\ \tfrac{4}{4+\|v\|}. \tag{4}$$

   Does the algorithm still converge with this new retraction? Compare both $R_v(x)$ and $R_v^{\text{new}}(x)$ by drawing the evolution of the objective function and the norm of the Riemannian gradient on a plot. What happens if you instead use a wrong retraction, e.g., $R_v^{\text{wrong}}(x) = x + v$? Does the algorithm still converge? Why?

Part 2: In the second part of the project, you will aim at recovering original images from the second dataset X_{part2} based on corrupted ones. We will use the graph learned in the first part of the

[1] To do this, you can create a new child class of `Sphere` in Python in which you only implement the retraction method. This way, your new class will inherit from all the properties of `Sphere` and hence will be compatible with pyManopt.

project to regularize the reconstruction process. Let $L^{(c)} = n(Q_\perp M^{(c)})(Q_\perp M^{(c)})^\top$ be the solution of (3) for $c \in \{R, G, B\}$ that you obtained in question 2 with $k = 30$ using the `TrustRegions` algorithm.

6. Implement with PyManopt the low-rank inpainting problem:

$$\min_{\hat{x}^{(c)} \in \mathbb{R}_r^{32\times 32}} \sum_{(s,t)\in\Omega} (y_{s,t}^{(c)} - \hat{x}_{s,t}^{(c)})^2 + \alpha \mathrm{vec}(\hat{x}^{(c)})^\top L^{(c)} \mathrm{vec}(\hat{x}^{(c)}), \tag{5}$$

   where $y^{(c)} \in \mathbb{R}^{32\times 32}$ is the channel $c \in \{R, G, B\}$ of the corrupted image $y \in \mathbb{R}^{3\times 32\times 32}$, and $\mathbb{R}_r^{32\times 32}$ is the manifold of rank-r 32×32 matrices.[2] The set Ω is the mask containing the pixel coordinates that have not been corrupted and $\alpha \geqslant 0$ is the graph regularization parameter.

7. For each corrupted image of X_{part2} and each channel $c \in \{R, G, B\}$, solve (5) with $\alpha = 2$, and rank parameter $r = 8$ by running `SteepestDescent`. Observe the impact of the graph regularization term by evaluating, for each image x in X_{part2}, the signal-to-noise ratio

$$\mathrm{SNR}(x, \hat{x}) := 10 \log_{10} \left(\frac{\sum_{c\in\{R,G,B\}} \|x^{(c)}\|_F^2}{\sum_{c\in\{R,G,B\}} \|x^{(c)} - \hat{x}^{(c)}\|_F^2} \right)$$

   obtained with $\alpha = 0$ and $\alpha = 2$. The higher the SNR, the better. Discuss your results.

8. Try several ranks and several values of $\alpha > 0$ on 5 corrupted images of your choice. How does the SNR evolve for each reconstructed image with respect to r and α? Which parameter has the strongest impact on the reconstructed image?

9. While, in the first part of the project, you relied on Burer-Monteiro factorizations to impose an upper bound on the rank of the Laplacian, in this second part, you directly imposed the rank by working on the manifold $\mathbb{R}_r^{32\times 32}$. Compare these two approaches conceptually on several aspects (differences, similarities, strengths, weaknesses,...). Moreover, take a step back on the project and motivate the introduction of low-rank constraints in the two cases.[3]

4 Practical information

- *Deadline:* **Sunday 15th of March 2026, 23:59**, in the "Project 1" activity on Moodle.
- *Material:* This assignment and one Jupyter Notebook to complete. The dataset should be downloaded via the link given above.

[2] While no expressions were seen yet for a metric and a retraction on this specific manifold, we propose you to rely on PyManopt, which already implements all the geometrical tools you will need for this manifold.

[3] Please note that this question is intentionally open-ended and does not have a single correct answer.

- *Deliverables:* Per group, one report in PDF format **.pdf** and one completed Jupyter Notebook in **.ipynb** format. If your Notebook has dependencies with external files (such as images or Python files **.py**), then the latter should be delivered too.

- *Report:*
  - 5 pages maximum, images included (4 pages should be enough), bibliography not included.
  - The font size should be 11 (single column) or 10 (double columns).
  - Please structure your report **in three sections**: a brief introduction recalling the context and the goal of the project, the main part containing your answers, and a brief conclusion including a summary of your results. The sections should respectively represent around 10-75-15% of the document.
  - The report should be understandable for a person with the background of someone starting a master's degree as a civil engineer in applied mathematics.

- *Scientific approach:*
  - Theorems and results from the course involve assumptions, conditions, parameters, variables and constants. Be clear and rigorous when you use them.
  - Your experimental protocols should be cleverly designed, clearly explained and robust to noise. In particular, when randomness is involved, repeat your experiments a sufficient number of times to validate the consistency of your results.
  - A good analysis should always compare your theoretical expectations with your numerical results.

- *References:* You might be interested in looking up external resources (scientific articles, blogs, online implementations, ...). Cite them correctly and include a bibliography at the end of your report.

- *AI:* Using AI writing/coding assistant (such as ChatGPT) is allowed, but you have to add a section at the end of the report detailing in which way you used this tool.

- *Q&A:* A Q&A sessions will take place on **Friday 6th of March 2026, 10:45-12:45**, in BARB13 (instead of the exercise session). If you come to the session, make sure to first explain in around 5 minutes what you've already done and to have prepared your questions in advance.

- *Contact:* As a last resort, you can send your questions and remarks directly to the course assistants Bastien Massion (`bastien.massion@uclouvain.be`) and Nicolas Mil-Homens Cavaco (`nicolas.mil-homens@uclouvain.be`).

References

[1] Xiaowen Dong et al. "Laplacian matrix learning for smooth graph signal representation". In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2015, pp. 3736–3740. DOI: 10.1109/ICASSP.2015.7178669.

[2] Franco Scarselli et al. "The Graph Neural Network Model". In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80. DOI: 10.1109/TNN.2008.2005605.

[3] James Townsend, Niklas Koep, and Sebastian Weichwald. "Pymanopt: A Python Toolbox for Optimization on Manifolds using Automatic Differentiation". In: *Journal of Machine Learning Research* 17.137 (2016), pp. 1–5. URL: http://jmlr.org/papers/v17/16-177.html.